

Update 1 – Archive Retrieval

Step 1: Update the Main Workbook's `createIndexSheet`

Find where `finalIndexMatrix` and your headers are defined inside your main workbook's `createIndexSheet` function, and replace that block with this updated columns structure:

JavaScript

None

```
// 1. Update the headers to include our new Column 10 (J) for
Restore Actions

var finalIndexMatrix = [

  ["Active Operational Sheets Workspace", "", "", "", "",
  "External Drive Cold-Storage Archives", "", "", "", ""],

  ["Generated", new Date(), "", "", "", "Archive File ID",
  archiveId, "", "", ""],

  ["", "", "", "", "", "", "", "", "", "", ""],

  ["Section / Month", "Sheet Tab Name", "Workspace Link",
  "Visibility", "", "Archive Month", "Archive Sheet Name", "Link to
  Sheet", "Status", "Restore Action"]

];

// 2. Inside your archive parsing loop, update the data rows
assignment:

try {

  var archiveSs = SpreadsheetApp.openById(archiveId);

  archiveSs.getSheets().forEach(function(ash) {

    var ashName = ash.getName();
```

```

        var ashDate = extractFirstDateFromSheetName_(ashName);

        var archiveMonthDisplay = "";

        if (ashDate) {

            archiveMonthDisplay = Utilities.formatDate(ashDate,
Session.getScriptTimeZone(), "MMMM yyyy");

        }

        archiveRows.push([

            archiveMonthDisplay,

            ashName,

            '=HYPERLINK("https://docs.google.com/spreadsheets/d/' +
archiveId + '/edit#gid=' + ash.getSheetId() + '", "Open Archive
Tab")',

            ash.isSheetHidden() ? "Archived (Hidden)" : "Visible in
Cold Storage",

            " [Click cell to select, then run Restore]" // This acts
as our row-level button label

        ]);

    });

    // ... maintain existing sort rules ...

```

Then, look for where the column widths are set near the bottom of `createIndexSheet`, and make sure Column 10 (J) is allocated proper spacing:

JavaScript

None

```
sheet.getRange(1, 5, finalIndexMatrix.length,
1).setBackground("#EDED");

sheet.setColumnWidth(5, 30);

// Extends width rules across all 10 columns

[1, 2, 3, 4, 6, 7, 8, 9, 10].forEach(function(col) {
sheet.setColumnWidth(col, 160); });
```

Step 2: Upgrade `restoreSheetFromActiveIndexRow`

Now we modify the recovery function to automatically look at Column 7 (G) to find the sheet name, regardless of whether the user clicked the name itself or the custom `[Restore]` link in Column 10 (J).

Replace your existing restoration code block with this adaptive tracker:

JavaScript

None

```
/**

 * Restores a historic sheet tab from the external Cold-Storage
Archive

 * back into the active Master List workbook based on row
selections.

 *

 * Assign this function to ONE permanent floating icon badge at the
top of the Index sheet.

 */

function restoreSheetFromActiveIndexRow() {

  var ui = SpreadsheetApp.getUi();
```

```
var mainSs = SpreadsheetApp.getActiveSpreadsheet();

var indexSheet = mainSs.getSheetByName("Index");

var activeRange = mainSs.getActiveRange();


// Enforce Index sheet isolation boundary

if (!indexSheet || !activeRange ||
activeRange.getSheet().getSheetId() !== indexSheet.getSheetId()) {

    ui.alert("Selection Required", "Please click on a row within
the 'External Drive Cold-Storage Archives' grid selection table
first.", ui.ButtonSet.OK);

    return;
}


var row = activeRange.getRow();

var col = activeRange.getColumn();


// Hard stop if they are clicking headers or the local data side

if (row < 5 || col < 6) {

    ui.alert("Invalid Selection", "Please click anywhere on an
Archive entry row (Columns F through J) before requesting a
restore.", ui.ButtonSet.OK);

    return;
}
```

```

    // FORCE PATH RE-DIRECTION: Irrespective of whether they
highlight the sheet name or the

    // [Restore] action column, look up Column 7 (G) to capture the
true sheet identity string.

    var targetSheetName = String(indexSheet.getRange(row,
7).getValue() || "").trim();

    if (!targetSheetName || targetSheetName.indexOf("Open Archive")
=== 0 || targetSheetName.indexOf("Archive Sheet") === 0) {

        ui.alert("Empty Selection", "The chosen row index coordinate
does not contain a valid archived sheet identifier.",
ui.ButtonSet.OK);

        return;
    }

    // Prevent breaking an active local tab profile

    if (mainSs.getSheetByName(targetSheetName)) {

        ui.alert("Conflict Detected", "The sheet '" + targetSheetName +
"' already exists locally in this workbook. Please rename or delete
the local copy first.", ui.ButtonSet.OK);

        return;
    }

    var confirmation = ui.alert(

```

```
        "Confirm Sheet Retrieval",

        "Are you sure you want to retrieve '" + targetSheetName + "'
from cold storage and restore it as an active workspace tab?",

        ui.ButtonSet.YES_NO

    );

    if (confirmation !== ui.Button.YES) return;

    var archiveId = getArchiveSpreadsheetId_();

    var archiveSs;

    try {

        archiveSs = SpreadsheetApp.openById(archiveId);

    } catch (err) {

        ui.alert("Connection Failure", "Could not connect to the
Archive workbook destination link.", ui.ButtonSet.OK);

        return;

    }

    var archiveSourceSheet =
archiveSs.getSheetByName(targetSheetName);

    if (!archiveSourceSheet) {
```

```
    ui.alert("Not Found", "The sheet '" + targetSheetName + "' was  
not found inside the external archive spreadsheet database file.",  
ui.ButtonSet.OK);
```

```
    return;
```

```
}
```

```
try {
```

```
    mainSs.toast("Retrieving '" + targetSheetName + "' from archive  
drive...", "Data Transfer Running", 5);
```

```
    var restoredSheet = archiveSourceSheet.copyTo(mainSs);
```

```
    restoredSheet.setName(targetSheetName);
```

```
    if (typeof restoredSheet.showSheet === "function") {
```

```
        restoredSheet.showSheet();
```

```
}
```

```
    if (typeof positionSheetInGlobalSortOrder_ === "function") {
```

```
        positionSheetInGlobalSortOrder_(restoredSheet);
```

```
}
```

```
    // Re-run the local index layout framework pass to clear out  
the line item
```

```

        createIndexSheet();

        mainSs.setActiveSheet(restoredSheet);

        ui.alert("Success", "The sheet '" + targetSheetName + "' has
        been successfully restored from cold storage.", ui.ButtonSet.OK);

    } catch (err) {

        ui.alert("Transfer Error", "Failed to fully restore the
        archived sheet: " + err.message, ui.ButtonSet.OK);

    }

}

```

Update 2 – System Sheet Order

To ensure sheets don't have to be shifted or resorted after generation, we need to adjust the **quickSystemSetup** function.

By reorganizing this macro to run the **rebuildFormatDashboardDefaults** first, followed immediately by **setupSystemSheets** (which initializes your system and template tabs in their natural rank order), and finishing with the **createIndexSheet**, Google Sheets will naturally position every new tab into its correct slot on creation.

Update **quickSystemSetup**

Locate your **quickSystemSetup** function and replace it with this optimized, correctly sequenced block:

JavaScript

```

None

function quickSystemSetup() {

```



```
// 1. REBUILD DEFINITIONS FIRST: Must have a pristine dashboard  
before system sheets look up order parameters
```

```
showQuickStartToast_("Quick System Setup [Step 1/5]: Rebuilding  
Format Dashboard defaults...");
```

```
rebuildFormatDashboardDefaults();
```

```
// 2. INITIALIZE AND SORT SYSTEM CHANNELS: Generates Timing,  
Quality, and base layers in explicit rank order
```

```
showQuickStartToast_("Quick System Setup [Step 2/5]: Initializing  
system sheets and organizing structural tabs...");
```

```
setupSystemSheets();
```

```
// 3. SECURE VERIFICATION MAPS: Populate your custom document  
properties maps for validation checks
```

```
showQuickStartToast_("Quick System Setup [Step 3/5]: Executing  
Dashboard Quality start-up verifications...");
```

```
runDashboardQualityStartUp();
```

```
// 4. GENERATE SMOKESCREEN DIAGNOSTICS: Run framework integrity  
checks on the raw canvas layers
```

```
showQuickStartToast_("Quick System Setup [Step 4/5]: Running  
framework architecture validation harness...");
```

```
runFrameworkSmokeValidation();
```

```
// 5. BUILD INDEX LAST: Compiles active and archive tabs onto the
front interface after all order positions are final

showQuickStartToast_("Quick System Setup [Step 5/5]: Building
centralized Index dashboard navigation matrix...");

createIndexSheet();

showQuickStartToast_("Quick System Setup complete. All system
architectures are correctly aligned.");
}
```

Why this fixes the sorting overhead:

Previously, building the Index before running the quality start-up verifications caused newly activated sheets to disrupt the visual layout. With this updated workflow sequence:

- The script reads the fresh dashboard profile data **before** creating any sheet objects.
- The structural engine paints the workspace tabs directly into their final destination index values.
- The **Index** dashboard runs last, mapping a perfectly stabilized sheet array in a single, lightning-fast rendering pass.

Update 3 – Protected Tail-Block Lock:

Clip 1: Imperenetrable Protected Tail Block & Order Alignment

This completely locks your sorting hierarchy and forces **Banners**, **Archive - Demo P**, and **Monthly Change** out of the templates block, while establishing a hard floor for the core diagnostics pages.

Locate your `getGlobalSheetSortRankByName_` function and replace it with this version:

JavaScript

None

```
function getGlobalSheetSortRankByName_(sheetName) {  
    sheetName = String(sheetName || "").trim();  
  
    if (sheetName === INDEX_SHEET) return 5;  
  
    // Tiers 1-10: Strict requested active operational and import  
    sequence  
  
    if (sheetName.indexOf(MASTER_LIST_PREFIX) === 0) return 10;  
  
    if (sheetName.indexOf(MONTHLY_CHANGE_REPORT_PREFIX) === 0) return  
20;  
  
    if (sheetName === DISENROLLED_EXCLUSION_SHEET ||  
/^Disenrolled\b/i.test(sheetName)) return 30;  
  
    if (sheetName.indexOf("Raw Data ") === 0 &&  
sheetName.indexOf("Raw Data - ") === -1) return 40;  
  
    if (sheetName.indexOf(BANNER_PREFIX) === 0 ||  
sheetName.indexOf(BANNER_REPORT_ALT_PREFIX) === 0) return 50;  
  
    if (sheetName.indexOf(CARE_PLAN_DUE_PREFIX) === 0 ||  
sheetName.indexOf(CARE_PLAN_DUE_DATE_ALT_PREFIX) === 0) return 60;  
  
    if (sheetName.indexOf(UNLOCKED_PREFIX) === 0 ||  
sheetName.indexOf("Unlocked Care Plan") === 0) return 70;  
  
    if (sheetName.indexOf("Raw Data - Banners") === 0 ||  
sheetName.indexOf("Raw Data - Banner Report") === 0) return 80;  
  
    if (sheetName.indexOf("Raw Data - CP Due") === 0 ||  
sheetName.indexOf("Raw Data - Care Plan Due") === 0) return 90;  
  
    if (sheetName.indexOf("Raw Data - Unlock CP") === 0 ||  
sheetName.indexOf("Raw Data - Unlocked") === 0) return 100;  
}
```

```
// Clean handling of standalone structural tabs to sit before
standard framework components

if (sheetName === DEMO_P_ARCHIVE_SHEET) return 140;

// Mid-Block Catch: Orphan or miscellaneous sheets/templates go
here

if (sheetName.indexOf("Template - ") === 0 ||
sheetName.indexOf("Template") !== -1) return 180;

if (sheetName === SHEET_TYPE.DEMO_P || sheetName ===
DEMO_P_PREFIX) return 190;

// HARD UNYIELDING TAIL-BLOCK LOCK: System diagnostics are
permanently fixed at the bottom

// and cannot be bypassed, intersected, or mixed up by custom
scripts or template additions.

if (sheetName === "Framework Timing Report" || sheetName ===
RFF_TIMING_SHEET) return 900;

if (sheetName === "Dashboard Quality Report" || sheetName ===
RFF_TEST_DASHBOARD_SHEET) return 910;

if (sheetName === RFF_DASHBOARD_SHEET) return 920;

if (sheetName === RFF_BASE_TEMPLATE_NAME) return 950;

return 145;

}
```

Update 4 – Timeframe Date-Lock Only

This removes the structural clock seconds string, ensuring that your metadata parameters utilize clear calendar coordinates.

In **createActiveDemoPFromRawData_** (Bottom block update):

JavaScript

None

```
sheet.getRange("A1").setValue(sheetDef.reportTitle || "Demo P");

// Explicit Prompt Month Date-Locks
if (monthParts) {

sheet.getRange("B2").setValue(monthParts.firstDay).setNumberFormat(
    "m/d/yyyy");

sheet.getRange("D2").setValue(monthParts.lastDay).setNumberFormat("m/d/yyyy");

}

// Stamped Date Value Only - Bypasses dynamic live time tracking clocks

var targetDateString = monthParts ?
Utilities.formatDate(monthParts.lastDay,
    Session.getScriptTimeZone(), "MM/dd/yyyy") : formatDateDisplay_(new Date());

sheet.getRange("E2").setValue("Date Updated: " + targetDateString)
```

```
.setFontFamily("Arial")  
  
.setFontSize(10)  
  
.setFontStyle("italic")  
  
.setHorizontalAlignment("left");
```

In updateDisenrolledExclusionReportDates_:

JavaScript

None

```
function updateDisenrolledExclusionReportDates_(sheet, monthParts)  
{  
  
    if (!sheet) return;  
  
    const startDate = monthParts && monthParts.firstDay ?  
monthParts.firstDay : getTodayLocalDate_();  
  
    const endDate = monthParts && monthParts.lastDay ?  
monthParts.lastDay : startDate;  
  
  
    sheet.getRange("A1").setValue(DISENROLLED_EXCLUSION_SHEET);  
  
    sheet.getRange("A2").setValue("Date");  
  
  
  
    sheet.getRange("B2").setValue(startDate).setNumberFormat("m/d/yyyy")  
);  
  
    sheet.getRange("C2").setValue("to").setHorizontalAlignment("center")  
);  
  
  
  
    sheet.getRange("D2").setValue(endDate).setNumberFormat("m/d/yyyy");
```

```

// Timeframe Date update block only

var targetDateString = monthParts ?
Utilities.formatDate(monthParts.lastDay,
Session.getScriptTimeZone(), "MM/dd/yyyy") : formatDateDisplay_(new
Date());

sheet.getRange("E2").setValue("Date Updated: " +
targetDateString)

    .setFontFamily("Arial")

    .setFontSize(10)

    .setFontStyle("italic")

    .setHorizontalAlignment("left");
}

```

Update 5 – Master List Index Month Group

This fixes the month parsing behavior and strips out the broken "Unassigned Month" layout.

Replace the center block of `createIndexSheet` (lines 1358–1377 in your source file) with this loop script:

JavaScript

None

```

// --- CONSOLIDATED CHRONOLOGICAL MONTH BLOCK ENGINE ---

var rawImportedSheets = getIndexSheetNamesInMonthSortOrder_(ss,
function(sheetName) {

```

```

return sheetName.indexOf("Raw Data") === 0 ||

    sheetName.indexOf(BANNER_PREFIX) === 0 ||

    sheetName.indexOf(BANNER_REPORT_ALT_PREFIX) === 0 ||

    sheetName.indexOf(CARE_PLAN_DUE_PREFIX) === 0 ||

    sheetName.indexOf(CARE_PLAN_DUE_DATE_ALT_PREFIX) === 0 ||

    sheetName.indexOf(UNLOCKED_PREFIX) === 0 ||

    sheetName.indexOf("Unlocked Care Plan") === 0;

});

var importGroups = {};

rawImportedSheets.forEach(function(name) {

    var sheetDate = extractFirstDateFromSheetName_(name);

    // Build an internal key to ensure consistent month group
mapping

    var groupKey = sheetDate ? Utilities.formatDate(sheetDate,
Session.getScriptTimeZone(), "MM.yy") : null;

    if (!groupKey) {

        // Secondary fallback lookup via text tokens if explicit date
stamps are missing

        var tokens = name.split(" ");

        var endToken = tokens[tokens.length - 1];

        if (/^\d{1,2}\.\d{2}$/.test(endToken)) groupKey = endToken;

    }

```



```
    if (!groupKey) return; // Discards incomplete formatting
    structures gracefully
```

```
    if (!importGroups[groupKey]) importGroups[groupKey] = [];
    importGroups[groupKey].push(name);
  });
```

```
  // Render month group blocks in descending order (Newest Month
  arrays at the top)
```

```
  Object.keys(importGroups).sort(function(a, b) { return
  b.localeCompare(a); }).forEach(function(groupKey) {
```

```
    var sectionLabel = "Imported Data - " + groupKey;
```

```
    if (importGroups[groupKey].length > 0) {
```

```
      var sampleDate =
      extractFirstDateFromSheetName_(importGroups[groupKey][0]);
```

```
      if (sampleDate) {
```

```
        sectionLabel = "Imported Data - " +
        Utilities.formatDate(sampleDate, Session.getScriptTimeZone(), "MMMM
        yyyy");
```

```
      }
```

```
    }
```

```
  // Sort items within this month according to your strict
  10-tier hierarchy order
```

```

importGroups[groupKey].sort(function(a, b) {
    return getGlobalSheetSortRankByName_(a) -
getGlobalSheetSortRankByName_(b);

});

addIndexSection_(sectionLabel, importGroups[groupKey]);

});

// --- END CONSOLIDATED ENGINE ---

```

Update 6 – Format Dashboard Section

This stops long section labels from clipping on the main setup console.

Locate your `writeDashboardDefaultsFast_` function and add this line near line 512, right after the range strategy logic:

JavaScript

None

```

sheet.getRange(1, 1, rows.length, width)

    .setFontFamily("Arial")

    .setFontColor("#000000")

    .setFontSize(10)

    .setHorizontalAlignment("left")

    .setVerticalAlignment("middle")

    .setWrapStrategy(SpreadsheetApp.WrapStrategy.CLIP);

```

```
// TEXT OVERFLOW SHIELD: Enforces overflow rules on title blocks
and subheaders

sheet.getRange(1, 1, Math.min(rows.length, 3),
width).setWrapStrategy(SpreadsheetApp.WrapStrategy.OVERFLOW);

if (sectionRanges.length) {

sheet.getRangeList(sectionRanges).setWrapStrategy(SpreadsheetApp.WrapStrategy.OVERFLOW);

}
```

Update 7 – Format Dashboard Change Tracking

To transform the changelog from a broad, section-level tracker into a highly precise, cell-level audit trail, we need to update two core areas of your monolithic script:

1. **Re-engineer the Changelog Engine (`updateFormatDashboardChangelog_`):** It must now unpack the data arrays, compare them column-by-column, and write explicit differential rows (`Timestamp | Section Changed | Column Changed | Previous Value | New Value`) into Section G.
2. **Upgrade the Dashboard Start-up Pipeline (`runDashboardQualityDashboardVerificationSections_`):** It needs to scan that newly structured changelog and dynamically paint any modified cells with your requested `#f3ffc7` highlight color.

Here are the exact code clips to update these sections in your script:



Clip 1: The Cell-Level Differential Changelog Engine

Locate your `updateFormatDashboardChangelog_` function and replace it entirely with this precision cell-differential engine. It breaks down every column edit and writes the exact structural changes to your requested headers.

JavaScript

None

```
function updateFormatDashboardChangelog_(dashboardSheet, timing) {  
    const props = PropertiesService.getDocumentProperties();  
  
    let logRows =  
getDashboardQualitySectionRows_(RFF_DASHBOARD_CHANGELOG_KEY);  
  
    // Enforce your exact requested header layout structure  
  
    if (!logRows || logRows.length === 0) {  
        logRows = [ ["Timestamp", "Section Changed", "Column Changed",  
"Previous Value", "New Value", "", ""]];  
    }  
  
    const sectionsToTrack = [  
        RFF_SECTION_GLOBAL,  
        RFF_SECTION_TITLE_ROWS,  
        RFF_SECTION_SHEETS,  
        RFF_SECTION_BEHAVIORS,  
        RFF_SECTION_SYSTEM_SURFACES,  
        RFF_SECTION_COLUMNS,  
        RFF_SECTION_HEADERS  
    ];  
  
    let changed = false;
```

```
const now = new Date();

sectionsToTrack.forEach(function(sectionTitle) {

    try {

        const currentRows = readDashboardSectionRows_(dashboardSheet,
sectionTitle);

        const propKey = RFF_QA_SECTION_PROP_PREFIX + "TRACKER_DATA_"
+ computeStableHash_(sectionTitle);

        const rawPrevious = props.getProperty(propKey);

        // Store current state instantly for the next compilation
comparison pass

        props.setProperty(propKey, JSON.stringify(currentRows));

        if (!rawPrevious) return; // Skip initial run baseline setup

        const previousRows = JSON.parse(rawPrevious);

        // Map headers for the targeted section to identify which
specific column changed

        const sectionHeader =
getDefaultDashboardQualityDetailHeader_({ title: sectionTitle });

        // Deep cell-by-cell matrix evaluation loop
```

```
for (let r = 0; r < currentRows.length; r++) {  
    const currRow = currentRows[r] || [];  
    const prevRow = previousRows[r] || [];  
    const maxCols = Math.max(currRow.length, prevRow.length);  
  
    for (let c = 0; c < maxCols; c++) {  
        const currVal = currRow[c] !== undefined ?  
String(currRow[c]).trim() : "";  
  
        const prevVal = prevRow[c] !== undefined ?  
String(prevRow[c]).trim() : "";  
  
        if (currVal !== prevVal) {  
            const columnName = sectionHeader[c] || "Column " + (c +  
1);  
  
            // Push precise cell modifications directly to the top  
of the changelog stack  
  
            logRows.splice(1, 0, [  
                now,  
                sectionTitle,  
                columnName,  
                prevVal || "(blank)",  
                currVal || "(blank)",
```

```

        "",
        ""

    ]);

    changed = true;
}

}

}

} catch (err) {

    logBestEffortWarning_("Cell differential changelog failed for
" + sectionTitle + ": " + err.message);

}

});

if (changed) {

    const trimmedLog = [logRows[0]].concat(logRows.slice(1, 100));
    // Maintain a 100-row scrolling audit window

    saveDashboardQualitySectionRows_(RFF_DASHBOARD_CHANGELOG_KEY,
trimmedLog, { deferSheetWrite: true });

    if (timing) markFrameworkStep_(timing, "Dashboard Quality
Section G cell-differential changelog updated");

}

return changed;

}

```

Clip 2: Dynamic Cell Highlight Injection

To make the **Format Dashboard** sheet automatically highlight modified fields with `#f3ffc7`, we need to map the changelog entries back to their coordinates on the dashboard canvas during the start-up validation routine.

Locate your `runDashboardQualityDashboardVerificationSections_` function and replace it with this upgraded version:

JavaScript

None

```
function runDashboardQualityDashboardVerificationSections_(timing,
qualitySheet) {

    const ss = SpreadsheetApp.getActiveSpreadsheet();

    const quality = qualitySheet ||
ensureDashboardQualityReportSheet_();

    const dashboard = ss.getSheetByName(RFF_DASHBOARD_SHEET);

    if (!dashboard) {

        const missingRows = [

            ["Verification Area", "Verification Item", "Expected Value",
"Actual Value", "Status", "Severity", "Quality Notes"],

            ["Format Dashboard", "Sheet Exists", RFF_DASHBOARD_SHEET,
"Missing", "FAIL", "Critical", "Create or rebuild Format Dashboard
before template creation."],

        ];

        saveDashboardQualitySectionRows_(RFF_DASHBOARD_VERIFY_GLOBAL_KEY,
missingRows, { deferSheetWrite: true });

    }

}
```



```
        replaceDashboardQualitySectionRows_(quality,  
getDashboardQualitySectionTitleForKey_(RFF_DASHBOARD_VERIFY_GLOBAL_  
KEY),  
buildTimestampedDashboardQualitySectionRows_(RFF_DASHBOARD_VERIFY_G  
LOBAL_KEY, missingRows));  
  
        return missingRows;  
  
    }  
  
}
```

```
const dashboardIndex = buildDashboardSectionIndex_(dashboard);
```

```
// 1. RUN THE CELL DIFFERENTIAL LOG: Captured array deltas are  
logged out first
```

```
const changelogUpdated =  
updateFormatDashboardChangelog_(dashboard, timing);
```

```
const sectionPayloads = [  
  
    { key: RFF_DASHBOARD_VERIFY_GLOBAL_KEY, rows:  
collectFormatDashboardGlobalInputVerificationRows_(dashboardIndex),  
label: "Section A Global Inputs and Section B Title Rows verified"  
},  
  
    { key: RFF_DASHBOARD_VERIFY_SHEETS_KEY, rows:  
collectFormatDashboardSheetDefinitionVerificationRows_(dashboardInd  
ex), label: "Section C Sheet Definitions verified" },  
  
    { key: RFF_DASHBOARD_VERIFY_BEHAVIORS_KEY, rows:  
collectFormatDashboardSheetBehaviorVerificationRows_(dashboardIndex  
) , label: "Section D Sheet Behaviors verified" },  
  
]
```

```

    { key: RFF_DASHBOARD_VERIFY_COLUMNS_KEY, rows:
collectFormatDashboardColumnDefinitionVerificationRows_(dashboardIndex), label: "Section F Column Definitions verified" },

    { key: RFF_DASHBOARD_VERIFY_HEADERS_KEY, rows:
collectFormatDashboardSheetHeaderVerificationRows_(dashboardIndex),
label: "Section G Sheet Headers verified" },

    { key: RFF_DASHBOARD_CHANGELOG_KEY, rows:
getDashboardQualitySectionRows_(RFF_DASHBOARD_CHANGELOG_KEY),
label: "Section G Format Dashboard Changelog verified" }

];

const sectionWrites = [];

sectionPayloads.forEach(function(item) {

    const previousRows = getDashboardQualitySectionRows_(item.key);

    const changed = !dashboardQualityRowsEqualValues_(previousRows,
item.rows) || (item.key === RFF_DASHBOARD_CHANGELOG_KEY &&
changelogUpdated);

    saveDashboardQualitySectionRows_(item.key, item.rows, {
deferSheetWrite: true });

    if (changed) {

        sectionWrites.push({

            title: getDashboardQualitySectionTitleForKey_(item.key),

            rows:
buildTimestampedDashboardQualitySectionRows_(item.key, item.rows)

        });

```

```

    }

});

if (sectionWrites.length) {
    replaceDashboardQualitySectionsRows_(quality, sectionWrites);
    markFrameworkStep_(timing, "Dashboard Quality Sections A-G
batch written");
}

// 2. DYNAMIC CELL HIGHLIGHT ENGINE: Scan recent log metrics and
paint modified cells on Format Dashboard

try {
    const logs =
getDashboardQualitySectionRows_(RFF_DASHBOARD_CHANGELOG_KEY);

    if (logs.length > 1) {
        const highlightRanges = [];

        // Review the latest changes within this validation cycle
loop
        for (let i = 1; i < Math.min(logs.length, 15); i++) {
            const logEntry = logs[i];
            const sectionTitle = logEntry[1];
            const columnName = logEntry[2];

```

```
const newValue = logEntry[4];

if (!sectionTitle || !columnName) continue;

const sectionBounds = getDashboardSectionBounds_(dashboard,
sectionTitle);

if (!sectionBounds) continue;

const sectionHeaders =
dashboard.getRange(sectionBounds.headerRow, 1, 1,
dashboard.getLastColumn()).getValues()[0];

const colIndex = sectionHeaders.indexOf(columnName);

if (colIndex === -1) continue;

// Scan the section row block to identify where the edited
value lives

const sectionDataRange =
dashboard.getRange(sectionBounds.dataStartRow, 1,
sectionBounds.dataRowCount, dashboard.getLastColumn());

const sectionValues = sectionDataRange.getValues();

for (let r = 0; r < sectionValues.length; r++) {

    if (String(sectionValues[r][colIndex]).trim() ===
String(newValue).trim()) {
```

```

        const targetRow = sectionBounds.dataStartRow + r;

        const targetCol = colIndex + 1;

        highlightRanges.push(rowColToA1_(targetRow,
targetCol));

        break;
    }
}

}

// Paint all modified coordinates instantly with a high-speed
RangeList call

if (highlightRanges.length > 0) {

dashboard.getRangeList(highlightRanges).setBackground("#f3ffc7");

    markFrameworkStep_(timing, "Highlighted " +
highlightRanges.length + " changed cells on Format Dashboard with
#f3ffc7");

}

}

} catch (err) {

    logBestEffortWarning_("Format Dashboard cell highlight paint
skipped: " + err.message);

}

```

```
    return true;
}
```

Update 8 – Dashboard Quality Expanded

Clip 1: Update Dashboard Section Constants

Replace your existing `RFF_DASHBOARD_QUALITY_SECTIONS` array constant block with this updated map:

JavaScript

None

```
const RFF_DASHBOARD_QUALITY_SECTIONS = [

    { key: RFF_DASHBOARD_VERIFY_GLOBAL_KEY, title: "SECTION A -  
GLOBAL INPUTS VERIFICATION" },

    { key: RFF_DASHBOARD_VERIFY_SHEETS_KEY, title: "SECTION B - SHEET  
DEFINITIONS VERIFICATION" },

    { key: RFF_DASHBOARD_VERIFY_BEHAVIORS_KEY, title: "SECTION C -  
SHEET BEHAVIOR VERIFICATION" },

    { key: RFF_DASHBOARD_VERIFY_COLUMNS_KEY, title: "SECTION D -  
COLUMN DEFINITIONS VERIFICATION" },

    { key: RFF_DASHBOARD_VERIFY_HEADERS_KEY, title: "SECTION E -  
SHEET HEADERS VERIFICATION" },

    { key: RFF_VALIDATION_SHEET, title: "SECTION F - TEMPLATE  
STRUCTURE & VALIDATION" },

    { key: RFF_DASHBOARD_CHANGELOG_KEY, title: "SECTION G - FORMAT  
DASHBOARD CHANGELOG" },
```

```

    { key: RFF_HEALTH_CHECK_SHEET, title: "SECTION H - FRAMEWORK
HEALTH CHECK" },

    { key: RFF_PERFORMANCE_SUMMARY_KEY, title: "SECTION I -
PERFORMANCE SUMMARY" },

    { key: RFF_MASTER_LIST_HEALTH_KEY, title: "SECTION J - RAW DATA
VALIDATION" }, // Title change from last step

    { key: RFF_CP_SYNC_DIAGNOSTICS_KEY, title: "SECTION K - CARE PLAN
SYNC VALIDATION" },

    { key: RFF_WORKFLOW_SYNC_VERIFICATION_KEY, title: "SECTION L -
WORKFLOW & SYNCHRONIZATION VERIFICATION" },


// NEW DEDICATED QUALITY BLOCKS

    { key: "Demo P Processing Validation", title: "SECTION M - DEMO P
QUALITY VALIDATION" },

    { key: "Disenrolled Exclusion Validation", title: "SECTION N -
DISENROLLED EXCLUSION VALIDATION" },

    { key: "Monthly Change Validation", title: "SECTION O - MONTHLY
CHANGE VALIDATION" },


// SHIFTED TAIL BLOCKS

    { key: "Summary", title: "SECTION P - SUMMARY" },

    { key: "Signoff", title: "SECTION Q - SIGNOFF" }

];

```

And ensure your detail header helper matches these new keys perfectly:

JavaScript

None

```
function getDefaultDashboardQualityDetailHeader_(section) {  
    const title = String(section && section.title || "");  
  
    if (title.indexOf("SECTION M - ") === 0) return ["Check Item",  
"Status", "Issue", "Quality Notes", "", "", ""];  
  
    if (title.indexOf("SECTION N - ") === 0) return ["Audit Item",  
"Status", "Issue", "Quality Notes", "", "", ""];  
  
    if (title.indexOf("SECTION O - ") === 0) return ["Layout Item",  
"Status", "Issue", "Quality Notes", "", "", ""];  
  
    // ... maintain existing fallbacks below ...  
  
    return ["Item", "Status", "Issue", "Quality Notes", "", "", ""];  
}
```

The New Pipeline Macro Engine

Replace the previous `runOperationalDataPipelineValidations_` function with this layout-driven engine. It verifies your datasets and flushes the results directly into their new standalone blocks.

JavaScript

None

```
/**  
  
    * Run advanced processing checks across the core operational  
    assets and write  
  
    * them to dedicated Section M, N, and O dashboard canvas  
    locations.  
  
    */  
  
function runOperationalDataPipelineValidations_(timing,  
qualitySheet) {
```



```

const ss = SpreadsheetApp.getActiveSpreadsheet();

const monthParts = getMonthDateParts_(new Date());

const quality = qualitySheet ||
ensureDashboardQualityReportSheet_();

const stagedSectionWrites = [];

// =====

// SECTION M - DEMO P QUALITY VALIDATION

// =====

const demoSheet = ss.getSheetByName(DEMO_P_PREFIX);

const demoRows = [["Check Item", "Status", "Issue", "Quality
Notes"]];

if (!demoSheet) {

    demoRows.push(["Demo P Processing", "FAIL", "Sheet Missing",
"The ongoing baseline Demo P sheet is completely missing."]);

} else {

    const dData = getDataValues_(demoSheet, HEADER_ROW,
DATA_START_ROW);

    const dHeaders = dData.headerMap || {};

    const flattenedContactIdx = dHeaders["Contact - 1"];

    let contactPopulatedCount = 0;

    dData.values.forEach(function(row) {

```

```
        if (flattenedContactIdx !== undefined &&
!isBlankCell_(row[flattenedContactIdx])) contactPopulatedCount++;

    });

    if (contactPopulatedCount === 0 && dData.values.length > 0) {

        demoRows.push(["Contact Flattening", "FAIL", "Flatten
Processing Failed", "Contact fields exist but zero rows underwent
flat record compression."]);

    } else {

        demoRows.push(["Contact Flattening", "PASS", "OK", "Flat
contact record compression verified active across row elements."]);

    }

    const hashIdx = dHeaders["Source Hash"];

    if (hashIdx === undefined || (dData.values.length > 0 &&
isBlankCell_(dData.values[0][hashIdx]))) {

        demoRows.push(["Cryptographic Metadata", "WARNING",
"Stale/Missing Hashes", "Source Hash tracking stamps are not
evaluating on participant updates."]);

    } else {

        demoRows.push(["Cryptographic Metadata", "PASS", "OK", "MD5
differential source tracking hashes are processing normally."]);

    }

}
```

```

    saveDashboardQualitySectionRows_("Demo P Processing Validation",
demoRows, { deferSheetWrite: true });

    stagedSectionWrites.push({

        title: "SECTION M - DEMO P QUALITY VALIDATION",

        rows: buildTimestampedDashboardQualitySectionRows_("Demo P
Processing Validation", demoRows)

    });


    // =====

    // SECTION N - DISENROLLED EXCLUSION VALIDATION

    // =====

    const exclusionSheet =
ss.getSheetByName(DISENROLLED_EXCLUSION_SHEET);

    const exRows = [["Audit Item", "Status", "Issue", "Quality
Notes"]];

    if (!exclusionSheet) {

        exRows.push(["Exclusion Audit", "FAIL", "Sheet Missing",
"Historical Disenrolled Exclusion repository tab not found."]);

    } else {

        const exData = getDataValues_(exclusionSheet, HEADER_ROW,
DATA_START_ROW);

        const exHeaders = exData.headerMap || {};

        const auditStampIdx =
exHeaders[DISENROLLED_EXCLUSION_ADDED_HEADER];

```

```
let badRowFlag = false;

exData.values.forEach(function(row) {

    if (auditStampIdx !== undefined &&
isBlankCell_(row[auditStampIdx])) badRowFlag = true;

});

if (badRowFlag) {

    exRows.push(["Audit Tracking Stamps", "WARNING", "Missing
Origin Stamps", "Some rows inside the exclusion sheet are missing
their tracking lineage metadata flags."]);

} else {

    exRows.push(["Audit Tracking Stamps", "PASS", "OK", "All
archived records contain clean traceability and origin tracking
flags."]);

}

}

saveDashboardQualitySectionRows_("Disenrolled Exclusion
Validation", exRows, { deferSheetWrite: true });

stagedSectionWrites.push({

    title: "SECTION N - DISENROLLED EXCLUSION VALIDATION",

    rows: buildTimestampedDashboardQualitySectionRows_("Disenrolled
Exclusion Validation", exRows)

});
```

```

// =====

// SECTION 0 - MONTHLY CHANGE VALIDATION

// =====

const mcrSheet = getMonthlySheetByPrefixAndDate_(ss,
MONTHLY_CHANGE_REPORT_PREFIX, monthParts.firstDay,
monthParts.lastDay);

const mcrRows = [["Layout Item", "Status", "Issue", "Quality
Notes"]];

if (!mcrSheet) {

    mcrRows.push(["Change Report Layout", "WARNING", "Report Tab
Missing", "No Monthly Change Report generated for the active month
context."]);

} else {

    const mcrValues = mcrSheet.getDataRange().getValues();

    let subsectionSubheadersFound = 0;

    mcrValues.forEach(function(row) {

        const cellText = String(row[0] || "").trim();

        if (["Enrollments", "Disenrollments", "Demographic Changes",
"Caseload Changes", "Contact Changes", "Banner Summary
Changes"].indexOf(cellText) !== -1) {

            subsectionSubheadersFound++;

        }

    });

```

```

    });

    if (subsectionSubheadersFound < 6) {

        mcrRows.push(["Subsection Framework", "FAIL", "Broken
Subheader Bands", "The Change Report layout is missing its key
operational subheader splits. Found only: " +
subsectionSubheadersFound + "/6."]);

    } else {

        mcrRows.push(["Subsection Framework", "PASS", "OK", "All 6
core change subheaders are correctly formatted and positioned."]);

    }

}

saveDashboardQualitySectionRows_("Monthly Change Validation",
mcrRows, { deferSheetWrite: true });

stagedSectionWrites.push({

    title: "SECTION 0 - MONTHLY CHANGE VALIDATION",

    rows: buildTimestampedDashboardQualitySectionRows_("Monthly
Change Validation", mcrRows)

});

// Flush all three new blocks onto the sheet canvas in one
optimized operation

if (stagedSectionWrites.length) {

    replaceDashboardQualitySectionsRows_(quality,
stagedSectionWrites);

```

```
}  
  
}
```

Make sure to add a call to `runOperationalDataPipelineValidations_(timing, qualitySheet)` inside your master function (`runDashboardQualityFull()`) so that these new distinct blocks print perfectly during every dashboard refresh pass.

Clip A: Update the Master Execution Grid

Locate your master dashboard workflow runner function, `runDashboardQualityFull`, and find the array named `sectionsToProcess`. Update that array map to include the three new validation keys so they flush out during a unified run pass:

JavaScript

None

```
const sectionsToProcess = [  
  
  { key: RFF_HEALTH_CHECK_SHEET, title: "SECTION H - FRAMEWORK  
HEALTH CHECK", rows:  
getDashboardQualitySectionRows_(RFF_HEALTH_CHECK_SHEET) },  
  
  { key: RFF_PERFORMANCE_SUMMARY_KEY, title: "SECTION I -  
PERFORMANCE SUMMARY", rows:  
getDashboardQualitySectionRows_(RFF_PERFORMANCE_SUMMARY_KEY) },  
  
  { key: RFF_MASTER_LIST_HEALTH_KEY, title: "SECTION J - RAW  
DATA VALIDATION", rows:  
getDashboardQualitySectionRows_(RFF_MASTER_LIST_HEALTH_KEY) },  
  
  { key: RFF_CP_SYNC_DIAGNOSTICS_KEY, title: "SECTION K - CARE  
PLAN SYNC VALIDATION", rows:  
getDashboardQualitySectionRows_(RFF_CP_SYNC_DIAGNOSTICS_KEY) },  
  
  { key: RFF_WORKFLOW_SYNC_VERIFICATION_KEY, title: "SECTION L  
- WORKFLOW & SYNCHRONIZATION VERIFICATION", rows:  
getDashboardQualitySectionRows_(RFF_WORKFLOW_SYNC_VERIFICATION_KEY)  
},  
]
```

```

// NEW REGISTRATION INJECTION HOOKS

    { key: "Demo P Processing Validation", title: "SECTION M -
DEMO P QUALITY VALIDATION", rows:
getDashboardQualitySectionRows_("Demo P Processing Validation") },

    { key: "Disenrolled Exclusion Validation", title: "SECTION N
- DISENROLLED EXCLUSION VALIDATION", rows:
getDashboardQualitySectionRows_("Disenrolled Exclusion Validation")
},

    { key: "Monthly Change Validation", title: "SECTION O -
MONTHLY CHANGE VALIDATION", rows:
getDashboardQualitySectionRows_("Monthly Change Validation") },


// SHIFTED EXTENSION TRAILERS

    { key: "Summary", title: "SECTION P - SUMMARY", rows:
summaryRows },

    { key: "Signoff", title: "SECTION Q - SIGNOFF", rows:
signoffRows.slice(1) }

];

```

Clip B: Enforce Custom Column Labels

Locate the helper function **normalizeDashboardQualityIssueValue_** or look directly at **normalizeDashboardQualityRowsForSection_**. Ensure that your letter-mapping conditional branch recognizes sections **M**, **N**, and **O** to properly slice data arrays:

JavaScript

None

```
const letter = getDashboardQualitySectionLetter_(section);
```



```

const output = [governedHeader];

for (let r = 1; r < input.length; r++) {
    const row = input[r] || [];

    let mapped;

    if (letter === "M") mapped = [row[0] || "", row[1] || "",
row[2] || "", row[3] || ""];

    else if (letter === "N") mapped = [row[0] || "", row[1] ||
"", row[2] || "", row[3] || ""];

    else if (letter === "O") mapped = [row[0] || "", row[1] ||
"", row[2] || "", row[3] || ""];

    // ... maintain existing structural letter mappings (A
through L, then P and Q) ...

```

Update 9 – Dashboard Quality Section J

Update - Raw Data Validation

This re-allocates **Section J** to target your live formatted **Raw Data** sheet instead of the Master List, confirming both primary row allocations and checking that the raw banner data sync successfully matches up fields.

Locate `runDashboardQualityMasterListHealthCheck_` (or your Section J engine) and replace it with this:

JavaScript

None

```
function runDashboardQualityRawDataValidation_() {

    const ss = SpreadsheetApp.getActiveSpreadsheet();

    const monthParts = getMonthDateParts_(new Date());

    const rawSheet = getCurrentRawDataSheet_(monthParts);

    const bannerSheet = getCurrentBannersSheet_(monthParts);


    const rows = [
        ["Validation Item", "Status", "Issue", "Quality Notes"]
    ];


    if (!rawSheet) {

        rows.push([
            "Raw Data Target", "FAIL", "Sheet Missing", "Active formatted Raw Data sheet not found for current month context."
        ]);

        saveDashboardQualitySectionRows_("Raw Data Validation", rows);

        return rows;

    }


    const rawData = getDataValues_(rawSheet, HEADER_ROW, DATA_START_ROW);

    const rawPmrIdx = getPMRIndex_(rawData.headerMap);

    const primaryIdx = rawData.headerMap["Primary PMR Row"];


    // 1. Validate Primary PMR Assignments

    if (primaryIdx === undefined || rawPmrIdx === -1) {
```

```
    rows.push(["Primary PMR Assignment", "FAIL", "Schema Missing",  
"Primary PMR Row column or PMR header is missing from Raw Data  
layout."]);
```

```
  } else {
```

```
    let primaryCount = 0;
```

```
    const seenPmr = new Set();
```

```
    let multiPrimaryCount = 0;
```

```
    rawData.values.forEach(function(row) {
```

```
      const pmr = normalizePMR_(row[rawPmrIdx]);
```

```
      if (!pmr) return;
```

```
      if (isPrimaryPMRRowValue_(row[primaryIdx])) {
```

```
        primaryCount++;
```

```
        if (seenPmr.has(pmr)) multiPrimaryCount++;
```

```
        seenPmr.add(pmr);
```

```
      }
```

```
    });
```

```
    if (multiPrimaryCount > 0) {
```

```
      rows.push(["Primary PMR Assignment", "FAIL", "Duplicate  
Primaries", "Detected " + multiPrimaryCount + " instances where a  
single PMR has multiple 'Yes' rows."]);
```

```
    } else if (primaryCount === 0 && rawData.values.length > 0) {
```

```

        rows.push(["Primary PMR Assignment", "WARNING", "No Primaries
Flags", "Raw Data rows exist but zero records are flagged as
Primary PMR Row = Yes."]);

    } else {

        rows.push(["Primary PMR Assignment", "PASS", "OK", "Primary
row assignment logic is fully active; mapped " + seenPmr.size + "
unique primary participant flags."]);

    }

}

// 2. Validate Banner Sync Integrity on Raw Data

if (!bannerSheet) {

    rows.push(["Banner Sync Sync Check", "WARNING", "Missing
Monthly Banner Sheet", "Cannot cross-verify Banner columns because
the formatted monthly Banners tab is missing."]);

} else {

    const bannerHeaders = ["Safety - 2 Person", "Wanderer",
"Interpreter Needed", "Fall Risk", "DPOA or Guardian Active",
"Palliative Care"];

    let mappedCheckCount = 0;

    let syncDiscrepancyCount = 0;

    const bannerMap =
buildSourceMapByCompositeKeyForDemoPBanner_(bannerSheet,
HEADER_ROW, DATA_START_ROW, ["Participant PMR#", "Last Name",
"First Name"]);

```

```
if (bannerMap.size > 0 && rawPmrIdx !== -1) {

    const rawLastNameIdx = rawData.headerMap["Last Name"];
    const rawFirstNameIdx = rawData.headerMap["First Name"];

    rawData.values.forEach(function(row) {

        if (rawLastNameIdx === undefined || rawFirstNameIdx ===
undefined) return;

        if (primaryIdx !== undefined &&
!isPrimaryPMRRowValue_(row[primaryIdx])) return;

        const key = [normalizeKeyPart_(row[rawPmrIdx]),
normalizeKeyPart_(row[rawLastNameIdx]),
normalizeKeyPart_(row[rawFirstNameIdx])].join("|||");

        const sourceMatch = bannerMap.get(key);

        if (!sourceMatch) return;

        mappedCheckCount++;

        bannerHeaders.forEach(function(field) {

            const rawIdx = rawData.headerMap[field];

            if (rawIdx === undefined) return;

            const rawCell = String(row[rawIdx] ||
"").trim().toUpperCase());
```

```

        const sourceCell = String(sourceMatch[field] ||
        "").trim().toUpperCase();

        if (rawCell !== sourceCell) syncDiscrepancyCount++;

    });

});

}

if (syncDiscrepancyCount > 0) {

    rows.push(["Banner Sync Verification", "FAIL", "Sync
Discrepancies", "Detected " + syncDiscrepancyCount + " cell
mismatches between active Raw Data and the Banners import
sheet."]);

    } else if (mappedCheckCount === 0) {

        rows.push(["Banner Sync Verification", "WARNING", "Zero
Matching Profile Keys", "No participants could be cross-matched by
PMR + Name keys between Banners and Raw Data."]);

    } else {

        rows.push(["Banner Sync Verification", "PASS", "OK", "Banner
synchronization verified clean across " + mappedCheckCount + "
active participant profiles."]);

    }

}

saveDashboardQualitySectionRows_("Raw Data Validation", rows);

return rows;

```

```
}
```

Upgrade 10 – Dashboard Quality Section K Master List Updates

This changes **Section K** from a basic blueprint checklist into an absolute data check. It reads your active **Master List** sheet and verifies that the data fields have actually inherited content from the current Unlocked and Care Plan Due source sheets.

Update your Section K function to include this row comparison loop:

JavaScript

None

```
function runDashboardQualityCarePlanSyncDiagnostics_(timing) {

    const ss = SpreadsheetApp.getActiveSpreadsheet();

    const monthParts = getMonthDateParts_(new Date());

    const rows = [{"Diagnostic", "Status", "Issue", "Quality
Notes"}];

    // 1. Core Dependency Check (Original Logic)

    const requiredFunctions = ["syncMasterListFromUnlockedCarePlan_",
"syncMasterListFromCarePlanDue_", "formatCarePlanDueReport",
"formatUnlockedCarePlanReport"];

    requiredFunctions.forEach(function(name) {

        const exists = existsFunctionByName_(name);

        rows.push([name, exists ? "PASS" : "FAIL", exists ? "OK" :
"Missing function", exists ? "Dependency is available." : "Repair
missing script dependency."]);

    });

}
```

```
});
```

```
// 2. ADVANCED DATA MERGE CHECK: Scan active Master List cells  
for real sync content
```

```
const masterName = buildMonthlySheetName_(MASTER_LIST_PREFIX,  
monthParts.firstDay, monthParts.lastDay);
```

```
const masterSheet = ss.getSheetByName(masterName);
```

```
if (!masterSheet) {
```

```
    rows.push(["Master List Content Merge", "WARNING", "Sheet  
Missing", "Active Master List tab not found for this month context;  
content sync audit skipped."]);
```

```
    } else {
```

```
        const masterData = getDataValues_(masterSheet, HEADER_ROW,  
DATA_START_ROW);
```

```
        const mHeaders = masterData.headerMap || {};
```

```
        let totalRowsEvaluated = masterData.values.length;
```

```
        let unpopulatedUnlockedCount = 0;
```

```
        let unpopulatedDueCount = 0;
```

```
        const unlockedFields = ["IDT Meeting Date", "Care Plan Start  
Date"];
```



```
    const dueFields = ["Enrollment Date", "Last Care Plan", "Next  
Care Plan Due", "CP Type"];
```

```
    masterData.values.forEach(function(row) {  
        // Check Unlocked fields  
        unlockedFields.forEach(function(f) {  
            const idx = mHeaders[f];  
            if (idx !== undefined && isBlankCell_(row[idx]))  
unpopulatedUnlockedCount++;  
        });  
        // Check Care Plan Due fields  
        dueFields.forEach(function(f) {  
            const idx = mHeaders[f];  
            if (idx !== undefined && isBlankCell_(row[idx]))  
unpopulatedDueCount++;  
        });  
    });
```

```
    const totalPossibleChecks = totalRowsEvaluated *  
(unlockedFields.length + dueFields.length);
```

```
    const totalBlanks = unpopulatedUnlockedCount +  
unpopulatedDueCount;
```

```

    if (totalRowsEvaluated === 0) {

        rows.push(["Master List Content Merge", "WARNING", "Empty
Dataset", "Master List contains headers but zero data rows."]);

        } else if (totalBlanks === totalPossibleChecks &&
totalPossibleChecks > 0) {

            rows.push(["Master List Content Merge", "FAIL", "Sync
Processing Failed", "All Care Plan sync destination columns are
completely blank. The merge process failed or ran out of
sequence."]);

            } else if (totalBlanks > (totalPossibleChecks * 0.40)) { //
Flag if more than 40% of records fail to map

                rows.push(["Master List Content Merge", "WARNING", "High
Volume Mapping Gaps", "More than 40% of sync values are blank.
Verify source matching keys (Name/PMR). Missing Unlocked values: "
+ unpopulatedUnlockedCount + ", Missing Due values: " +
unpopulatedDueCount]);

            } else {

                rows.push(["Master List Content Merge", "PASS", "OK", "Merge
validation confirmed; sync data populated across active columns
(Audited cells: " + totalPossibleChecks + ", Blank variations: " +
totalBlanks + ")."]);

            }

        }

    }

    saveDashboardQualitySectionRows_(RFF_CP_SYNC_DIAGNOSTICS_KEY,
rows);

```

```
    if (timing) markFrameworkStep_(timing, "Dashboard Quality Section  
K verified with active content cell loops");  
  
    return rows;  
  
}
```

Upgrade 11 – Section M – Demo P Quality Validation

SECTION M – DEMO P QUALITY VALIDATION

Since **Demo P** is the central, persistent working engine where all your massive text transformations happen, this section explicitly audits its data consolidation success:

- **Contact Flattening Verification:** It scans the active cells in the **Contact - 1** column. If rows exist but are completely blank, it flags a **FAIL**, alerting you that your background loop failed to compress multi-line imported cell rows into flat, clean single-row participants.
- **Cryptographic Metadata Tracking:** It verifies the health of the **Source Hash** column. It checks that the MD5 comparison strings are being dynamically stamped on every row so your incremental updates don't break.

Section M: Demo P Quality Validation Script

JavaScript

None

```
/**  
  
 * Core validation logic for Section M: Demo P Quality  
Validation.  
  
 * Audits contact record flattening states and cryptographic  
row hash integrity.  
  
 */  
  
function runDashboardQualityDemoPValidation_() {
```

```

    const ss = SpreadsheetApp.getActiveSpreadsheet();

    const rows = [["Check Item", "Status", "Issue", "Quality
Notes"]];

    const demoSheet = ss.getSheetByName(DEMO_P_PREFIX);

    if (!demoSheet) {

        rows.push(["Demo P Processing", "FAIL", "Sheet Missing",
"The ongoing baseline Demo P sheet is completely missing."]);

        saveDashboardQualitySectionRows_("Demo P Processing
Validation", rows, { deferSheetWrite: true });

        return rows;
    }

    const dData = getDataValues_(demoSheet, HEADER_ROW,
DATA_START_ROW);

    const dHeaders = dData.headerMap || {};

    // 1. Audit Contact Flattening Processes

    const flattenedContactIdx = dHeaders["Contact - 1"];

    if (flattenedContactIdx === undefined) {

        rows.push(["Contact Flattening", "FAIL", "Missing Schema
Column", "The core 'Contact - 1' destination column is missing
from Demo P."]);

    } else {

```

```

    let contactPopulatedCount = 0;

    dData.values.forEach(function(row) {

        if (!isBlankCell_(row[flattenedContactIdx]))
        contactPopulatedCount++;

    });

    if (contactPopulatedCount === 0 && dData.values.length >
0) {

        rows.push(["Contact Flattening", "FAIL", "Flatten
Processing Failed", "Contact fields exist but zero active
participant rows underwent multi-line contact compression."]);

    } else {

        rows.push(["Contact Flattening", "PASS", "OK", "Flat
contact record compression verified active across " +
contactPopulatedCount + " participant profiles."]);

    }

}

// 2. Audit MD5 Row Hash Tracking

const hashIdx = dHeaders["Source Hash"];

if (hashIdx === undefined) {

    rows.push(["Cryptographic Metadata", "FAIL", "Missing
Tracker Column", "The structural 'Source Hash' audit tracking
column is missing from Demo P."]);

} else {

```

```

    let blankHashCount = 0;

    dData.values.forEach(function(row) {

        if (isBlankCell_(row[hashIdx])) blankHashCount++;

    });

    if (blankHashCount > 0) {

        rows.push(["Cryptographic Metadata", "WARNING", "Stale/Missing Hashes", "Detected " + blankHashCount + " participant rows operating without active change tracking stamps."]);

    } else if (dData.values.length === 0) {

        rows.push(["Cryptographic Metadata", "WARNING", "No Data Present", "Demo P is currently empty; verification baseline skipped."]);

    } else {

        rows.push(["Cryptographic Metadata", "PASS", "OK", "MD5 differential source tracking hashes are fully evaluated across all rows."]);

    }

}

saveDashboardQualitySectionRows_("Demo P Processing Validation", rows, { deferSheetWrite: true });

return rows;

}

```

Upgrade – 12 Section N – Disenrolled Exclusion Validation

— SECTION N – DISENROLLED EXCLUSION VALIDATION

This section acts as a strict audit gatekeeper for your **Disenrolled Exclusion** sheet to verify that no participant records are lost or dropped when stripped out of the main active loops:

- **Traceability Lineage Audit:** It checks every single row inside the exclusion repository for a valid origin stamp in the **Added to Disenrolled Exclusion** column. If it finds a row missing its stamp, it flags a **WARNING** so you can clean up entries that lack a timestamped paper trail.

— Section N: Disenrolled Exclusion Validation Script

JavaScript

```
None
/**
 * Core validation logic for Section N: Disenrolled Exclusion
 * Validation.
 * Verifies audit trail presence and historic data traceability.
 */
function runDashboardQualityDisenrolledExclusionValidation_() {
  const ss = SpreadsheetApp.getActiveSpreadsheet();

  const rows = [["Audit Item", "Status", "Issue", "Quality
Notes"]];

  const exclusionSheet =
ss.getSheetByName(DISENROLLED_EXCLUSION_SHEET);
```

```

    if (!exclusionSheet) {

        rows.push(["Exclusion Audit", "FAIL", "Sheet Missing",
"Historical Disenrolled Exclusion repository tab not found."]);

        saveDashboardQualitySectionRows_("Disenrolled Exclusion
Validation", rows, { deferSheetWrite: true });

        return rows;

    }

    const exData = getDataValues_(exclusionSheet, HEADER_ROW,
DATA_START_ROW);

    const exHeaders = exData.headerMap || {};

    const auditStampIdx =
exHeaders[DISENROLLED_EXCLUSION_ADDED_HEADER];

    // 1. Audit Origin Lineage Stamps

    if (auditStampIdx === undefined) {

        rows.push(["Audit Tracking Stamps", "FAIL", "Missing Stamp
Schema", "The required tracking header '" +
DISENROLLED_EXCLUSION_ADDED_HEADER + "' is missing from the
sheet."]);

    } else {

        let missingStampCount = 0;

        exData.values.forEach(function(row) {

            if (isBlankCell_(row[auditStampIdx])) missingStampCount++;

```



```

    });

    if (missingStampCount > 0) {

        rows.push(["Audit Tracking Stamps", "WARNING", "Missing
Origin Stamps", "Detected " + missingStampCount + " historical rows
operating without an origin tracking lineage stamp."]);

        } else if (exData.values.length === 0) {

            rows.push(["Audit Tracking Stamps", "PASS", "Empty
Repository", "The historical exclusion list is clear; no current
entries to evaluate."]);

        } else {

            rows.push(["Audit Tracking Stamps", "PASS", "OK", "All
historical records contain clean, uncorrupted traceability origin
stamps."]);

        }

    }

    saveDashboardQualitySectionRows_("Disenrolled Exclusion
Validation", rows, { deferSheetWrite: true });

    return rows;

}

```

Upgrade 13 – Section 0 – Monthly Change Validation



SECTION 0 – MONTHLY CHANGE VALIDATION

This section checks the visual structure and data completeness of your Monthly Change Report to make sure your end-of-month layout arrays compiled successfully:

- **Banding Framework Integrity:** It scans column A of the report for your 6 core subsection markers (*Enrollments, Disenrollments, Demographic Changes, Caseload Changes, Contact Changes, Banner Summary Changes*). If it finds fewer than 6, it drops a **FAIL**, letting you know that a background compilation error broke your reporting structure.



Section 0: Monthly Change Validation Script

JavaScript

None

```
/**
```

```
 * Core validation logic for Section 0: Monthly Change Validation.
```

```
 * Audits report structural layout configuration states and  
subheader preservation.
```

```
 */
```

```
function runDashboardQualityMonthlyChangeValidation_() {
```

```
    const ss = SpreadsheetApp.getActiveSpreadsheet();
```

```
    const monthParts = getMonthDateParts_(new Date());
```

```
    const rows = [ ["Layout Item", "Status", "Issue", "Quality  
Notes" ] ];
```

```

    const mcrSheet = getMonthlySheetByPrefixAndDate_(ss,
MONTHLY_CHANGE_REPORT_PREFIX, monthParts.firstDay,
monthParts.lastDay);

    if (!mcrSheet) {

        rows.push(["Change Report Layout", "WARNING", "Report Tab
Missing", "No Monthly Change Report generated for the active month
workspace."]);

        saveDashboardQualitySectionRows_("Monthly Change Validation",
rows, { deferSheetWrite: true });

        return rows;
    }

    // 2. Audit Structural Subsection Framework Presence

    const mcrValues = mcrSheet.getDataRange().getValues();

    const requiredBands = ["Enrollments", "Disenrollments",
"Demographic Changes", "Caseload Changes", "Contact Changes",
"Banner Summary Changes"];

    const foundBands = [];

    mcrValues.forEach(function(row) {

        const cellText = String(row[0] || "").trim();

        if (requiredBands.indexOf(cellText) !== -1 &&
foundBands.indexOf(cellText) === -1) {

            foundBands.push(cellText);

```

```
    }

    });

    if (foundBands.length < requiredBands.length) {

        const missingBands = requiredBands.filter(function(band) {
return foundBands.indexOf(band) === -1; });

        rows.push(["Subsection Framework", "FAIL", "Broken Subheader
Bands", "The Monthly Change Report layout is missing structural
blocks. Missing: " + missingBands.join(", ")]);

    } else {

        rows.push(["Subsection Framework", "PASS", "OK", "All 6
mandatory chronological change subsection subheaders are correctly
formatted and drawn."]);

    }

    saveDashboardQualitySectionRows_("Monthly Change Validation",
rows, { deferSheetWrite: true });

    return rows;

}
```

Upgrade 14 - Monthly Change Report Caseload History



The Targeted Caseload Differential String Engine

Replace the `buildMonthlyChangeReportRow_` function in your script with this version:

JavaScript

None

```
function buildMonthlyChangeReportRow_(sourceRow, sourceHeaders,
reportHeaders, changedColumns, dateIndexes, previousItem,
previousHeaderMap) {

    const output = sourceRow.slice(0, sourceHeaders.length);

    // Pad to match report target layout width
    while (output.length < reportHeaders.length) {
        output.push("");
    }

    const changeIdx = reportHeaders.indexOf("Columns With Change");
    if (changeIdx !== -1) {
        if (changedColumns && changedColumns.size > 0) {
            const detailedChangeStrings = [];

            const prevValues = previousItem && previousItem.values ?
previousItem.values : [];
```

```
// Strict list of allowed Caseload Columns requested for  
historical differential mapping
```

```
const caseloadFilter = [  
  "Caseload - Social Work",  
  "Caseload - RN",  
  "Caseload - PCP",  
  "Caseload - HCC",  
  "Caseload - Activities",  
  "Caseload - OT",  
  "Caseload - PT",  
  "Caseload - RD",  
  "Caseload - Supervising MD"  
];
```

```
Array.from(changedColumns).sort().forEach(function(columnName) {
```

```
  // CONDITION CHECK: Is this specifically one of our  
  caseload targets?
```

```
  if (caseloadFilter.indexOf(columnName) !== -1) {
```

```
    const prevColIdx = previousHeaderMap ?  
previousHeaderMap[columnName] : undefined;
```

```
    let previousValueDisplay = "";
```

```
        if (prevColIdx !== undefined && prevColIdx !== -1 &&
prevValues.length > 0) {

            const rawPrevValue = prevValues[prevColIdx];

            previousValueDisplay =
displayValueForReport_(rawPrevValue);

        }

        if (previousValueDisplay !== "") {

            detailedChangeStrings.push(columnName + " -- " +
previousValueDisplay);

        } else {

            detailedChangeStrings.push(columnName + " -- (blank)");

        }

    } else {

        // Fallback Strategy: Standard columns are appended
        exactly by name text only

        detailedChangeStrings.push(columnName);

    }

});

    output[changeIdx] = detailedChangeStrings.join(", ");

} else {

    output[changeIdx] = "";
```

```
    }  
}  
  
    return convertMonthlyChangeReportDateValues_(output,  
reportHeaders, dateIndexes);  
}
```